

Decision Trees and Random Forests

Introduction

Decision trees are a popular machine learning method often used for classification and regression tasks. As the name suggests, they are built in a tree-like structure where each node represents a test on an input feature, each resulting branch from the node represents the outcome of the test, and the leaf node represents a class label or a predicted value.

Now, what makes decision trees so popular? A major advantage of decision trees is their interpretability that is easy to understand, even for people who have no background in statistics. The tree structure represents the flow of a decision-making process, making it simple to understand the logic behind each step. Decision trees can handle both numerical and categorical features as well, making them ideal for analysing different types of data. Decision trees also have the ability to handle missing data by making a logical first guess and fine tuning it later on. This makes decision trees very effective at handling high-dimensional data and complex relationships between features and target variables. Decision trees are also scale invariant and hence, will not be affected by non-parameterization.

However, there are also some disadvantages to decision trees. Traditional decision trees implement splits that are parallel to the axes, which may not be optimal for all datasets, especially those with complex decision boundaries (Bishop, C. M., 2006). Furthermore, the splits in decision trees are hard, where each input space corresponds to just one leaf node. This may be problematic in regression, where smooth, continuous functions are desired but trees produce piecewise, discontinued functions instead.

A common extension of decision trees are random forests, a type of ensemble learning where multiple trees are combined to improve performance by aggregating their predictions and outputting the mode. Random forests often address the biggest issue that decision trees face -- overfitting.

Decision Trees

At each node of a decision tree, an important question to ask is: which feature should we use to split the data and if the data is continuous, what is the respective threshold to choose? Since decision trees work by recursively partitioning the input space based on features and thresholds, it is important to establish a framework to systematically split the data. Gain measures are essential for determining the best feature to split on at each node as they quantify the improvement in the purity of the data after a split, aiming to create branches that group similar data points together. There are various gain measures used to evaluate the quality of splits, some of which include the decrease in training error, information gain, which is based on entropy, and the Gini index.

To utilise these splitting criteria effectively, there are many tree constructing algorithms to choose from. Common algorithms like ID3 employ a greedy approach, selecting the best split at each node based on the aforementioned gain measures. However, greedy approaches may lead to

suboptimal trees as decisions are made based on local optima and cannot guarantee a global optimum. Furthermore, algorithms like the ID3 have a big issue: the returned trees tend to be very large (Shalev-Shwartz, S. et al., 2014). Such trees might have a low bias but are likely to have very large variance since decision trees with unlimited depth can overfit training data and lead to poor generalisation performance. The sample complexity of learning a decision tree with k leaves is proportional to k and learning the entire class of decision trees, which has an infinite VC dimension, can hence lead to overfitting (Shalev-Shwartz, S. et al., 2014).

Pruning

Some solutions to overfitting include implementing a limit on the number of iterations in the algorithm, leading to a tree with a bounded number of nodes. More commonly however, pruning techniques are used to combat overfitting. Pruning balances the accuracy against model complexity, aiming for a smaller tree with comparable performance. The pruning process involves cutting off the tree at certain nodes and hence combining their corresponding regions into larger ones. Decision trees can be pruned before or after they are built -- pre-pruning or post-pruning. Pre-pruning involves making a decision of whether to split at a node at all, depending on the increase in predictive accuracy, most commonly quantified by some gain measure. However, there's always a chance to prematurely cut a branch that might have a significant increase in predictive accuracy down the line, but leaving no way to find out since it was cut early, leading to suboptimal trees. Hence, pre-pruning is also considered greedy. Post-pruning however, mitigates that issue and is done after the tree is built, aiming to reduce the size of the tree while maintaining similar predictive accuracy. It is of course, the more computationally expensive of the 2 methods. It is usually performed by a bottom-up walk on the tree and each node might be replaced with one of its subtrees or with a leaf, based on some bound or estimate of true risk (Shalev-Shwartz, S. et al., 2014). A generic pruning procedure may proceed as the following:

Generic Tree Pruning Procedure
input: function $f(T, m)$ (bound/estimate for the generalization error of a decision tree T , based on a sample of size m), tree T . foreach node j in a bottom-up walk on T (from leaves to root): find T' which minimizes $f(T', m)$, where T' is any of the following: the current tree after replacing node j with a leaf 1. the current tree after replacing node j with a leaf 0. the current tree after replacing node j with its left subtree. the current tree after replacing node j with its right subtree. the current tree. let $T := T'$.

Generic Tree Pruning Procedure (Shalev-Shwartz, S. et al., 2014)

The algorithm uses a function to estimate the generalisation error and chooses a replacement option for each node that leads to the minimisation of said function. This proceeds iteratively for every node in the tree and results in a pruned tree that is less complex and less prone to

overfitting. Essentially, the algorithm tries to simplify the tree by replacing subtrees with single leaves or smaller subtrees, as long as these replacements don't significantly increase the generalisation error.

Random Forests

Random forests are a type of ensemble method where they combine many trees and output the class that is the mode of the classifications of all the individual trees -- that accounts for the 'forest' aspect. Now, the randomness arises as each individual tree is dependent on the values of a random vector that is sampled independently and with the same distribution for all trees in the forest (Breiman, L., 2001). A random vector θ_k is generated, independent of past random vectors $\theta_1, \dots, \theta_{k-1}$, but identically distributed, and a tree is built using the training set and θ_k , resulting in a classifier $h(x, \theta_k)$ where x is an input vector.

Random forests are often dubbed as the more accurate predictor in comparison to decision trees since it combats the problem of overfitting that comes from individual decision trees. It makes intuitive sense that as more trees exposed to different datasets or features combine, they tend to make less error overall. It is akin to having several students study for the same test using the same textbook and practice questions -- they are more likely to make the same mistakes compared to a group of students who have had varied materials and practice papers to study from. This intuition was vigorously quantified by showing that the generalisation error, a measure of how well the model can predict unseen data, converges as the number of trees in the forest grows, by the Strong Law of Large Numbers (Breiman, L., 2001). We will also find that an upper bound for the generalisation error can be derived in terms of 2 parameters -- how accurate each individual classifier is, the strength, and the dependence between the classifiers, the correlation (Breiman, L., 2001).

Now, there are 2 common ways of introducing randomness -- bagging and random feature selection, which works on randomising the dataset and the features respectively. The random vector θ could represent the random subset of the dataset, or a random subset of the features -- the nature and dimensionality of θ is dependent on its use when constructing a tree. We will see that this handmade randomness minimises correlation while still maintaining accuracy of the classifiers.

Convergence of Random Forests

Breiman (2001) has proved that the generalisation error of random forests converges, showing that random forests are effective in overcoming overfitting. He first defined the margin function as

$mg(X, Y) = av_k I(h_k(X) = Y) - \max_{j \neq Y} av_k I(h_k(x) = j)$ where $I(\cdot)$ is the indicator function. This measures the extent to which the average number of votes for the right class exceeds the average number of votes for any other class -- quantifying the confidence of the classification. He then defined the generalisation error in terms of the margin function:

$$PE^* = P_{X,Y}(mg(X, Y) < 0)$$

Then, for a large number of trees, it follows from the Strong Law of Large Numbers that as the number of trees increase, the generalisation error, PE^* converges to

$P_{X,Y}(P_\theta(h(X, \theta) = Y) - \max_{j \neq Y} P_\theta(h(X, \theta) = j) < 0)$, hence showing that overfitting is not a problem for random forests, as the forests grow.

Upper Bound for Generalisation Error

Breiman (2001) has also shown that how good a random forest is is dependent only on the accuracy of the classifiers, strength, and the dependence between the classifiers, correlation, i.e. the generalisation error can be upper bounded in terms of these 2 terms. An upper bound for the generalisation error was given by

$PE^* \leq \frac{\bar{\rho}(1 - s^2)}{s^2}$, where $\bar{\rho}$ is the mean value of the correlation between the classifiers and s represents the strength.

This bound suggests that a lower correlation and a higher strength will overall result in better generalisation.

Bagging

The randomness introduced to the forests has to minimise the correlation $\bar{\rho}$ while maintaining strength, s . The forests studied in Breiman (2001) have produced results that are comparable with AdaBoost and have the following characteristics:

- i) Its accuracy is as good as Adaboost and sometimes better.
- ii) It's relatively robust to outliers and noise.
- iii) It's faster than bagging or boosting.
- iv) It gives useful internal estimates of error, strength, correlation and variable importance.
- v) It's simple and easily parallelized.

Characteristics of Random Features (Breiman, L. 2001)

The randomness is often introduced by bagging and random feature selection. Bagging refers to bootstrap and aggregating, where bootstrap samples are new training sets obtained by drawing a random set from the original dataset, with replacement, and then aggregating the results of all the classifiers built upon each bootstrap sample. The trees are then built on this new training set using random feature selection. On average, about $\frac{1}{3}$ of the original data points are not included in each bootstrap sample -- these points are considered out-of-bag (OOB) for that specific tree. For each data point in the training set, the predictions are only made by the trees for which that data point is OOB, i.e. a data point is only used to evaluate the performance of trees that were not trained on that same data point. The OOB estimate is calculated by aggregating the OOB predictions across all data points and comparing them to the true labels. This provides an estimate of the forest's performance on unseen data without the need for a separate test set and

studies have shown that OOB estimates are often as accurate as testing using a separate test set (Breiman, L., 1996).

Random Feature Selection

Some predictor variables might get overemphasised in the model and lead to highly correlated trees which results in overfitting. Bagging is used to randomly sample the “rows”, the data points, and on the other hand, random feature selection is used to randomly select the “columns”, the predictors. It allows the model to better generalise -- now that the features are limited at each node, different decision trees will have access to different sets of features, introducing variability into the decision trees. There are a few ways to go about random feature selection -- random input selection, random linear combinations, etc. Breiman (2001) found that the simplest method was random input selection, denoted as Forest-RI, where a small group of predictor variables to split on are selected at random at each node. The tree is then grown without pruning. The results of Forest-RI are comparable to that of AdaBoost, and is much faster than AdaBoost and Bagging. A question we might now have would be, what would be the optimal number of features in that small group, denoted as F . Breiman (2001) found that this procedure is actually not sensitive to F -- the results of running Forest-RI with $F=1$ yielded similar results, with a test error within $\pm 1\%$ of that of the model with higher values of F .

Relation to AdaBoost

We all know that AdaBoost is also an ensemble method that combines weak learners to create a stronger predictor. It focuses on assigning higher weights to samples that were incorrectly classified in previous classifiers so that the next classifier will focus more on these mistakes. The final classification is a weighted combination of the classifications of all the weak learners, with the more accurate learners with higher weights. The weight distribution in AdaBoost is done systematically and does not seem random -- however, Breiman (2001) likens AdaBoost to a random forest. Breiman (2001) suggests that as more classifiers are trained, the weights assigned converge to a distribution and hence, the weights are just selected at random from said distribution. Thus, this similarity to random forests, and thus the convergence of its generalisation error, may explain why AdaBoost is not as prone to overfitting. On the other hand, we know from the lecture notes that AdaBoost's resistance to overfitting stems from the margin maximisation -- AdaBoost minimises test error even after achieving a training error of 0, leading to better generalisation.

However, even if AdaBoost's behaviour can be seen as random sampling from a distribution, it doesn't explain how the distribution is derived in the first place (Breiman, L. 2001), since the mechanism of AdaBoost is deterministic. The distribution of weights generated by AdaBoost is also tied to the specific training data, where different training data will lead to different weight distribution, a contrast to random forests where randomness is independent of the training set.

Application

Applications of decision trees and random forests are made easy with programming libraries like SKLearn on Python. By just calling a simple function, the libraries do all the work for us while we reap the results. To illustrate the mechanism however, I built a very simple decision tree to classify the chances that a Major League Baseball (MLB) team would make it to the play-offs using some batting statistics of the team and opposing teams, namely batting average (BA), on base percentage (OBP), slugging rate (SLG), opponent on base percentage (OOBP) and opponent slugging rate (OSLG). Typically, a higher BA, OBP, and SLG indicates good offence in a team and a lower OOBP and OSLG indicates good defence for a team. The core idea of this decision tree model is finding the right node to split the data on, i.e. the feature and the corresponding threshold, if it is numerical, which partitions the data into pure, homogenous groups. In order to find the feature and threshold that best separates the data, the information gain measure was used. Each feature was looped through while the entropy was calculated for each split and we chose that specific feature and threshold that minimises the entropy. For and while loops were used to recursively section the data using the information gain as a criteria until a stopping criteria is met — either the maximum depth, which I chose as 3, has been reached, or a sufficiently pure split has been obtained. Once a node can't be split any further, a leaf node with a label will be outputted. As a comparison, I ran 2 models using the DecisionTreeClassifier and RandomForestClassifier from SKLearn with the same parameters and found that all 3 models had the same accuracy of 78.1%.

Classification Report for Decision Tree Classifier:					Classification Report for Random Forest Classifier:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.86	0.97	0.91	203	0	0.85	0.97	0.90	203
1	0.68	0.30	0.41	44	1	0.57	0.18	0.28	44
accuracy			0.85	247	accuracy			0.83	247
macro avg	0.77	0.63	0.66	247	macro avg	0.71	0.58	0.59	247
weighted avg	0.83	0.85	0.82	247	weighted avg	0.80	0.83	0.79	247
Accuracy: 78.14%					Accuracy: 78.14%				

The lacklustre performance in the RandomForestClassifier might be attributed to the correlation among features, since the batting statistics are pretty much indicating the same thing -- how good the team is in offence or defence. As mentioned, how good random forests are at classification relies on the strength and correlation. With highly correlated features, the predictions may still be similar since each random subset of features would still be similar to another random subset of features, ultimately increasing correlation.

However, it still stands that random forests do much better at classification than decision trees for the aforementioned reasons.

Conclusion

Decision trees and random forests are essential tools in machine learning with complementary strengths and weaknesses. While decision trees offer easy interpretability due to their hierarchical structure based on a series of binary decisions, random forests provide better generalisation through ensemble learning and randomisation. By knowing the mechanism behind these classifiers, we are better equipped to build better and more robust models.

References

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning* (Michael Jordan, Jon Kleinberg, & Bernhard Schölkopf, Eds.). Springer Science+Business Media, LLC.

<http://research.microsoft.com/~cmbishop>

Breiman, L. (1996). *Bagging predictors*. *Machine Learning*, 24(2), 123–140.

<https://doi.org/10.1007/bf00058655>

Breiman, L. & Statistics Department, University of California. (2001). *Random forests*.

MA4270 Lecture Notes. (2024). *MA4270 Lecture Notes #7: Boosting* [Lecture notes].

Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.

<http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning>